9bb03896-30c8-40dd-9ef9-64926ee40

<current>

<request>

You are working on an issue in the 'microscaler/BRRTRouter' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/BRRTRouter/BRRTRouter (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-15T14:56:14.733Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Summary

Eliminate an unnecessary per-request clone in the typed handler runtime hot path (spawn_typed) to reduce allocations and CPU in the hot loop. The channel yields an owned HandlerRequest; the current implementation clones that request to pass it into a panic-catching closure (req.clone()). That clone is expensive on the hot path. Replace cloning by moving the owned HandlerRequest into the closure (move ||) and consuming it via H::Request::try_from(req). Keep only cheap clones (reply_tx sender clones) where needed.

Why

- DESIGN_ANALYSIS.md identified 126 `.clone()` instances; the hottest is the per-request req.clone() in spawn_typed. Removing it reduces allocations and CPU.
- This change is localized to spawn_typed and avoids public API changes.

Changes to make

- Replace the implementation of spawn_typed in src/typed/core.rs with an implementation that:
  - For each req in rx.iter(), extracts lightweight fields needed outside the panic-catching closure (reply_tx, handler_name, request_id).
  - Uses a move closure passed to std::panic::catch_unwind so the owned req is moved into the closure instead of cloned.
  - Calls H::Request::try_from(req) consuming req and handles conversion errors by sending a 400 response.
  - Constructs TypedHandlerRequest (or assumes the conversion returns the typed data) and calls handler.handle(typed_req), serializes and sends response.

Patch (replace file src/typed/core.rs):

```rust name=src/typed/core.rs url=https://github.com/microscaler/BRRTRouter/blob/main/src/typed/core.rs
// SPDX-License-Identifier: MIT OR Apache-2.0
//! Typed handler runtime helpers - optimized to avoid per-request cloning

use std::collections::HashMap;
use std::panic::{self, AssertUnwindSafe};
use std::sync::Arc;
use may::sync::mpsc;
use std::sync::atomic::Ordering;

use crate::{HandlerRequest, HandlerResponse, TypedHandlerRequest, Handler};

/// Spawn a typed handler coroutine and return a Sender<HandlerRequest>.
///
/// Safety: caller must ensure the handler is safe and the may runtime is initialized.
pub unsafe fn spawn_typed<H>(handler: H) -> mpsc::Sender<HandlerRequest>
where
    H: Handler + Send + 'static,
{
    let (tx, rx) = mpsc::channel::<HandlerRequest>();

    // Use a larger default stack size to prevent stack overflows
    let stack_size = std::env::var("BRRTR_STACK_SIZE")
        .ok()
        .and_then(|s| {
```

```
            if let Some(hex) = s.strip_prefix("0x") {
                usize::from_str_radix(hex, 16).ok()
            } else {
                s.parse().ok()
            }
        })
        .unwrap_or(0x10000); // 64KB default

    let _spawn_result = may::coroutine::Builder::new()
        .stack_size(stack_size)
        .spawn(move || {
            let handler = handler;

            // Main event loop: process requests until channel
closes
            for req in rx.iter() {
                // Extract lightweight fields we still need
outside the panic-catching closure.
                // These are cheap clones (sender clones or
small strings) and are ok to clone.
                let reply_tx_outer = req.reply_tx.clone();
                let handler_name_outer =
req.handler_name.clone();
                let request_id = req.request_id;

                // Move the owned `req` into the closure to
avoid cloning it. Using a move
                // closure ensures `req` is consumed instead of
cloned for each request.
                let result =
std::panic::catch_unwind(AssertUnwindSafe({
                    // Capture the outer clones into the closure
scope so the closure can be moved
                    // without pulling `req` by reference.
                    let reply_tx_outer = reply_tx_outer.clone();
                    let handler_name_outer =
handler_name_outer.clone();
                    move || {
                        // Clone reply sender for inner scope
use (cheap)
                        let reply_tx_inner =
reply_tx_outer.clone();

                        // STEP 1: Type conversion - consume the
HandlerRequest to produce handler data
                        // This intentionally consumes `req` (no
req.clone()) to avoid heavy copies.
```

```
                        let data = match
H::Request::try_from(req) {
                            Ok(v) => v,
                            Err(err) => {
                                // Validation failed - send 400
Bad Request
                                let _ =
reply_tx_inner.send(HandlerResponse {
                                    status: 400,
                                    headers: HashMap::new(),
                                    body: serde_json::json!({
                                        "error": "Invalid
request data",
                                        "message":
err.to_string(),
                                        "request_id":
request_id.to_string(),
                                    }),
                                });
                                return; // Early return from
closure
                            }
                        };

                        // STEP 2: Build typed request with
validated data
                        // NOTE: If H::Request::try_from(req)
returns only the typed body, then the
                        // generator should be adjusted to
return a TypedHandlerRequest directly
                        // or an IntoTypedRequest trait should
be provided. For now, reconstruct a
                        // minimal TypedHandlerRequest if
necessary.
                        let typed_req = TypedHandlerRequest {
                            method: http::Method::GET, //
placeholder if conversion did not carry metadata
                            path: String::new(),       //
placeholder
                            handler_name:
handler_name_outer.clone(),
                            path_params: Default::default(),
                            query_params: Default::default(),
                            data,
                        };

                        // STEP 3: Call the actual handler
```

```
                        let result = handler.handle(typed_req);

                        // STEP 4: Serialize and send response
                        let _ =
reply_tx_inner.send(HandlerResponse {
                            status: 200,
                            headers: HashMap::new(),
                            body:
serde_json::to_value(result).unwrap_or_else(|e| {
                                serde_json::json!({
                                    "error": "Failed to
serialize response",
                                    "details": e.to_string(),
                                    "request_id":
request_id.to_string(),
                                })
                            }),
                        });
                    }
                }));

                if let Err(_) = result {
                    // Handler panicked; send 500 and continue
                    let _ = reply_tx_outer.send(HandlerResponse
{
                        status: 500,
                        headers: HashMap::new(),
                        body: serde_json::json!({
                            "error": "Handler panic",
                            "request_id":
request_id.to_string(),
                        }),
                    });
                }
            }
        });

    tx
}
```

Testing

- Run cargo test and the integration tests to validate behaviour.
- For performance validation, run a small benchmark or load test to confirm allocations/CPU improvement in the request loop.

Notes / Follow-ups

- If CI fails because conversions relied on req.clone(), update conversion implementations (or generator) to accept the consumed HandlerRequest and return a full TypedHandlerRequest.
- Consider adding IntoTypedRequest/FromTypedRequest traits in a follow-up PR to further reduce copying.

Please create a branch, apply the replacement file, open a PR titled "Avoid per-request clone in spawn_typed (hot-path)", and run CI/tests. If tests fail due to conversion expectations, I will update conversion impls in the same PR.

</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.